

AI-Powered Internal Developer Framework

IT Internal Planning Meeting

สร้าง Self-Service Platform ที่ Maintenance Deploy ง่ายได้

Zero Config • On-Premise • AI-Powered

สิงหาคม 2026

The Problem: Maintenance = Developer

Maintenance พัฒนา app เอง แต่มี pain points:

- ❌ ต้องส่ง code ให้ IT deploy → รอ 1-3 วัน
- ❌ IT สั่งให้แก้ config, environment → วุ่นวาย
- ❌ ไม่มี self-service deployment
- ❌ เปรียบเทียบกับ Vercel/Render/Cloudflare/NeonDB/Supabase ที่ zero config
- ❌ งบประมาณในการพัฒนาไม่เพียงพอ
- ❌ มีขั้นตอนมาก ต้องผ่านหลาย approval

Quote จาก Maintenance:

"ถ้าเอาไป deploy บน Vercel เสร็จใน 5 นาที แต่ต้องรอ IT 3 วัน"

Objective

เป้าหมายหลัก:

1. ลดขั้นตอน — จาก 10+ ขั้นตอน → 3 ขั้นตอน (Code → Push → Deploy)
2. ลดเวลา — จาก 3 วัน → 15 นาที
3. ลดค่าใช้จ่าย — ใช้ infrastructure ที่มีอยู่ให้คุ้มค่าที่สุด
4. เพิ่มประสิทธิภาพ — Maintenance พัฒนาและ deploy ได้เอง 100%
5. **Data Privacy** — ควบคุม code และ data ภายในองค์กร

KPIs:

- Deployment Frequency: 10x/สัปดาห์
- Lead Time for Changes: < 15 นาที
- Change Failure Rate: < 5%

Benefit

สำหรับ Maintenance:

-  Deploy เองได้ ไม่ต้องรอ IT
-  ทดสอบได้ทันที (Preview Environment)
-  สร้าง database เองได้ (Self-service)
-  AI ช่วยเขียนโค้ดและ docs
-  ลดเวลา deploy 99% (3 วัน → 15 นาที)

สำหรับ IT:

-  ลด ticket จาก Maintenance
-  ควบคุม security และ compliance
-  Audit trail ครบถ้วน
-  ใช้ infrastructure ที่มีอยู่ให้คุ้มค่า

สำหรับองค์กร:

-  ลด Shadow IT (80% → 0%)
-  เพิ่ม innovation speed
-  ควบคุม data privacy 100%
-  Cost avoidance: ฿2.7M ใน 3 ปี
-  ROI > 4,000%

สำหรับ Developer Experience:

-  Zero config — ใช้งานเหมือน Vercel
-  Self-service — ทำเองได้ทันที
-  AI-powered — LLM ช่วยทุกขั้นตอน

🤔 Why Not Just Use Vercel/Render?

External Cloud Platforms:

- ❌ **Data Privacy** — ไม่เอา code/data ออกนอกองค์กร
- ❌ **Compliance** — ต้องควบคุม audit trail
- ❌ **Cost** — ค่าใช้จ่ายต่อเดือนสูง
- ❌ **Control** — ควบคุม infrastructure ไม่ได้

Our Solution:

- ✅ **On-Premise** — ทุกอย่างอยู่ในองค์กร
- ✅ **Data Privacy** — ควบคุมได้ 100%
- ✅ **Zero Config** — ใช้งานเหมือน Vercel
- ✅ **AI-Powered** — LLM ช่วยทุกขั้นตอน

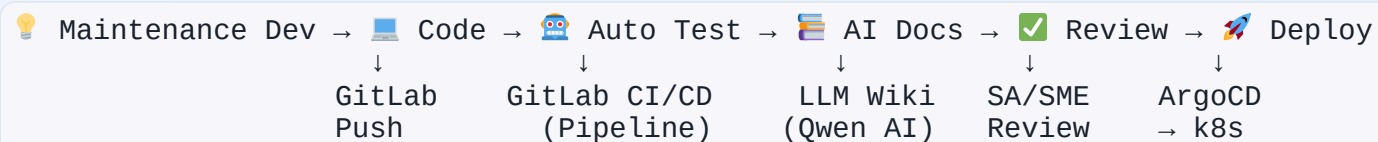
Our Vision: "Vercel-like, but On-Premise"

สิ่งที่ Maintenance จะได้:

-  **Push code** → **Deploy อัตโนมัติ** (15 นาที)
-  **Zero config** — ไม่ต้อง config infrastructure
-  **Self-service database** — สร้างได้ทันที (เหมือน NeonDB)
-  **AI-powered** — LLM ช่วยเขียนโค้ด, สร้าง docs, review code
-  **Data privacy** — ใช้ LLM ภายในองค์กร (Alibaba Qwen)

IT = Enabler, ไม่ใช่ Gatekeeper

Architecture Overview



Key Components:

- **GitLab** — Git + CI/CD pipeline
- **ArgoCD** — GitOps deploy to k8s
- **Kubernetes** — Container orchestration
- **LiteLLM + Qwen** — On-premise LLM
- **Vault** — Secrets management

Key Features

1. Self-Service Deployment

- Push → Deploy (เหมือน Vercel)
- ไม่ต้องรอ IT

2. Zero Config Database

- สร้าง database อัตโนมัติ (เหมือน NeonDB)
- Auto connection strings

3. Preview Environments

- ทุก MR ได้ environment ของตัวเอง
- ทดสอบก่อน merge

4. AI-Powered

- LLM ช่วยเขียนโค้ด
- Auto-generate documentation
- Auto code review

5. Data Privacy

- ใช้ LLM ภายในองค์กร
- ควบคุม data 100%

6. Governance

- Audit trail คน
- ตรวจสอบได้

 Tech Stack

Layer	Tool	Purpose
Git + CI	GitLab	Git server + CI/CD pipeline
CD	ArgoCD	GitOps deploy to k8s
Orchestration	Kubernetes	Container orchestration
Database	PostgreSQL + pgvector	AI embeddings, RAG
DB Operator	CloudNativePG	PostgreSQL automation
LLM Gateway	LiteLLM	Cache, log, route LLM calls
LLM	Alibaba Qwen	Token Plan (on-premise)
Secrets	Vault	API keys, credentials
Registry	Harbor	Container images
Monitoring	Prometheus + Grafana	Metrics, dashboards

Team Overview: IT = Enabler

หลักการ:

- **Maintenance = Developer** — พัฒนา app เอง, deploy เอง
- **IT = Platform Provider** — สร้าง platform ที่ใช้ง่าย

5 Teams:

1. **Maintenance** — Developer ที่พัฒนา app เอง
2. **SA** — ออกแบบ architecture, workflow
3. **AI-Eng** — LLM integration, auto-doc
4. **Dev** — Implementation, templates
5. **Infra&Security** — Infrastructure setup, security

Key Responsibilities:

- Review Function Spec
- Provide System & Module AI
- Review Code for AI
- Test Security

Goal: ทำให้ Maintenance deploy เองได้ 100% (ไม่ต้องรอ IT)



Maintenance Team (Developer)

Role: Developer ที่พัฒนา application เอง

Responsibilities:

- พัฒนา application (factory, back office, innovation)
- Push code ไป GitLab
- Deploy เองผ่าน self-service platform
- ให้อ feedback, requirements

Deliverables:

- ส่งมอบ Requirement Spec
- ส่งมอบ Function Spec ตาม Template Document Digital

Pain Points to Solve:

- ❌ ไม่ต้องรอ IT deploy
- ❌ ไม่ต้อง config infrastructure เอง
- ❌ ไม่ต้องส่ง ticket ขอ database, secrets

IT Must Deliver:

- ✅ Self-service deployment (push → deploy)
- ✅ Zero config database (เหมือน NeonDB)
- ✅ Auto secrets injection (เหมือน Vercel)



SA Team (Solution Architect)

Role: ឧបករណ៍ architecture, workflow, schema

Must Do:

- ឧបករណ៍ system architecture (GitLab → ArgoCD → k8s)
- ឧបករណ៍ wiki schema (LLM auto-doc structure)
- ឧបករណ៍ approval workflow (code review, wiki publish)
- ឧបករណ៍ RBAC model (factory vs backoffice vs user apps)

Review:

-  Review Requirement Spec
-  Review Functional Spec
-  Review Architecture Design
-  Review Workflow Design

Design Tasks:

- [] Architecture diagram (high-level + detailed)
- [] Wiki schema (markdown structure, metadata)
- [] Approval workflow (state machine)
- [] RBAC matrix (roles, permissions)

Deliverables: Architecture docs, workflow diagrams, RBAC matrix



AI-Eng Team

Role: LLM integration, auto-doc generation, MCP

Must Do:

- Integrate Alibaba Qwen 72B LiteLLM (on-premise)
- สร้าง LLM wiki generator (code → documentation)
- สร้าง MCP server template (ทุก app มี MCP)
- สร้าง auto PR reviewer (AI code review)
- Optimize LLM caching (ลด cost)

Design Tasks:

- [] LLM integration architecture (LiteLLM config)
- [] Wiki generation prompt engineering
- [] MCP server specification
- [] Auto PR review rules

Deliverables: LLM integration, wiki generator, MCP template, auto reviewer



Dev Team

Role: Implementation, templates, workflow

Must Do:

- ສ້າງ GitLab CI/CD templates (pipeline as code)
- ສ້າງ Kubernetes manifests (deployment, service, ingress)
- ສ້າງ database provisioning scripts (CloudNativePG)
- ສ້າງ preview environment automation
- Integrate Vault (secrets management)

Review:

- ☒ Review Source Code
- ☒ Review CI/CD Pipeline
- ☒ Review Kubernetes Manifests

Design Tasks:

- ☐ CI/CD pipeline templates (YAML)
- ☐ Kubernetes manifest templates
- ☐ Database provisioning workflow
- ☐ Preview environment lifecycle

Deliverables: CI/CD templates, k8s manifests, DB provisioning scripts

Infra&Security Team

Role: Infrastructure setup, security, monitoring

Must Do:

- Setup Kubernetes cluster (ใช้ server ที่มีอยู่)
- Setup GitLab + GitLab Runner
- Setup ArgoCD (GitOps)
- Setup Vault (secrets management)
- Setup Harbor (container registry)
- Setup Prometheus + Grafana (monitoring)
- Config network policies, RBAC

Design Tasks:

- [] Kubernetes cluster design (nodes, namespaces)
- [] Network topology (VLAN, firewall rules)
- [] Vault integration architecture
- [] Monitoring dashboard design

Deliverables: k8s cluster, GitLab, ArgoCD, Vault, monitoring

Team Collaboration Matrix

Collaboration Flow:

Team	ต้องรอจาก	ต้องส่งให้
Maintenance	N/A	Requirement Spec + Functional Spec → SA
SA	Requirement Spec (Maintenance)	Architecture + Review Spec → AI-Eng, Dev, Infra
AI-Eng	Architecture (SA)	LLM integration + AI Module → Dev
Dev	Architecture (SA), Infra (Infra)	Templates + Source Code Review → Maintenance
Infra	Architecture (SA)	Infrastructure → Dev, AI-Eng

Critical Path:

1. **Maintenance** → เตรียม Requirement & Functional Spec → ส่งให้ SA confirm
2. **SA** → Review Spec → ออกแบบ Architecture → ส่งให้ Dev, AI-Eng, Infra
3. **Infra** → Setup Infrastructure → ส่งให้ Dev, AI-Eng
4. **Dev** → สร้าง Templates → Review Source Code → ส่งให้ Maintenance



RACI Matrix

Activity	Maintenance	SA	AI-Eng	Dev	Infra
Requirement & Functional Spec	R/A	C	I	I	I
Review Spec	C	R/A	C	C	C
Architecture design	C	R/A	C	C	C
LLM integration	I	C	R/A	C	I
k8s setup	I	C	I	C	R/A
CI/CD templates	C	I	I	R/A	C
Review Source Code	C	C	C	R/A	I
Test Security	R	C	C	C	R/A

Legend: R = Responsible, A = Accountable, C = Consulted, I = Informed

Phase 1: Foundation (Month 1-2)

Goals:

- Setup infrastructure (k8s, GitLab, ArgoCD)
- Deploy pilot app แรก (Predictive Maintenance)
- **Maintenance deploy ง่ายได้ (ไม่ต้องรอ IT)**

Tasks by team:

- **Maintenance:** เตรียม Requirement Spec + Functional Spec
- **SA:** Review Spec → ออกแบบ architecture, wiki schema
- **AI-Eng:** Integrate Qwen LLM, สร้าง wiki generator prototype
- **Dev:** สร้าง CI/CD templates, Kubernetes manifests
- **Infra:** Setup k8s cluster, GitLab, ArgoCD, Vault

Review Milestones:

- ☒ Review Requirement & Functional Spec (Week 1)
- ☒ Review Architecture Design (Week 2)
- ☒ Review Infrastructure Setup (Week 4)

Deliverables:

- ☒ k8s cluster running
- ☒ GitLab + ArgoCD configured
- ☒ Pilot app deployed โดย Maintenance
- ☒ Baseline metrics recorded

Phase 2: Scale (Month 3-6)

Goals:

- เพิ่ม 5-10 apps
- วัด DORA metrics จริง
- **Zero config database working**

Tasks by team:

- **SA:** Review Architecture → ออกแบบ approval workflow, RBAC
- **AI-Eng:** สร้าง MCP server template, auto PR reviewer
- **Dev:** Review Source Code → สร้าง database provisioning, preview environments
- **Infra:** Config network policies, monitoring dashboards
- **Maintenance:** ให้อะ feedback, ทดสอบระบบ

Review Milestones:

- ☒ Review Workflow & RBAC Design (Month 3)
- ☒ Review Database Provisioning (Month 4)
- ☒ Review Preview Environments (Month 5)
- ☒ Review Security & Compliance (Month 6)

Deliverables:

- ☒ 5-10 apps deployed โดย Maintenance
- ☒ DORA metrics improved (deployment time < 15 นาที)
- ☒ Auto-doc generation working

Phase 3: Optimize (Month 7-12)

Goals:

- Optimize resource + automation
- สร้าง innovation culture
- **Maintenance deploy 100%**

Tasks by team:

- **SA:** Review & Optimize architecture based on feedback
- **AI-Eng:** Optimize LLM caching, and cost
- **Dev:** Review & Optimize CI/CD pipelines, and build time
- **Infra:** Right-sizing resources, auto-scaling
- **Maintenance:** แสร้ง success stories, เพิ่ม apps

Review Milestones:

- ☒ Review Performance Metrics (Month 8)
- ☒ Review Cost Optimization (Month 9)
- ☒ Review Automation Level (Month 10)
- ☒ Review ROI & Success Metrics (Month 12)

Deliverables:

- ☒ Resource utilization > 70%
- ☒ Deployment time < 15 นาที (เหมือน Vercel)
- ☒ Maintenance deploy 100%

! Challenge 1: Zero Config Experience

ปัญหา:

ทำให้ใช้ง่ายเหมือน Vercel/Render ยาก

ความท้าทาย:

- Auto-detect framework (Next.js, React, Node.js)
- Auto-config build settings
- Auto-inject environment variables
- Auto-provision database

วิธีแก้:

- ☒ สร้าง CI/CD templates ที่ auto-detect framework
- ☒ ใช้ CloudNativePG สำหรับ auto-provision database
- ☒ ใช้ Vault สำหรับ auto-inject secrets
- ☒ สร้าง preview environments อัตโนมัติ

Target: Push → Deploy ใน 15 นาที

! Challenge 2: Data Privacy with LLM

ปัญหา:

ต้องใช้ LLM แต่ไม่ส่ง data ออกนอกองค์กร

ความท้าทาย:

- Code อาจมี sensitive data
- Documentation อาจมีข้อมูลลับ
- ต้องควบคุม LLM calls 100%

วิธีแก้:

- ☒ ใช้ Alibaba Qwen ผ่าน LiteLLM (on-premise)
- ☒ LLM caching (ลด duplicate calls)
- ☒ Audit trail (ทุก LLM call)
- ☒ Network isolation (LLM อยู่ภายในองค์กร)

Target: Data privacy 100%

Immediate Actions (สัปดาห์นี้)

SA:

- ☐ Draft architecture diagram
- ☐ Draft wiki schema

AI-Eng:

- ☐ Setup LiteLLM + Qwen integration
- ☐ Test wiki generation prototype

Dev:

- ☐ Draft CI/CD pipeline templates
- ☐ Draft Kubernetes manifests

Infra:

- ☐ Inventory existing servers
- ☐ Draft k8s cluster design

Maintenance:

- ☐ List pain points (deployment, database, secrets)
- ☐ Select 2-3 pilot apps

Goal: พร้อมสำหรับ meeting กับ user สัปดาห์หน้า



Meeting with User (สัปดาห์หน้า)

Goals:

- Present framework to Maintenance
- Demo zero-config deployment (เปรียบเทียบกับ Vercel)
- Get feedback on requirements
- Confirm pilot projects

Agenda (90 นาที):

1. จุดเริ่มต้น: ไอเดียที่ดีสมควรได้รับการสนับสนุน (15 นาที)
2. Journey: จากไอเดียสู่ระบบจริง (20 นาที)
3. ทีมของเรา: ใครจะช่วยอะไรได้บ้าง (15 นาที)
4. AI ช่วยอะไรได้บ้าง (15 นาที)
5. เริ่มต้นอย่างไร (15 นาที)

Expected Outcome:

- ☒ Maintenance เห็นภาพและเข้าใจ
- ☒ Confirm pilot projects
- ☒ ตั้ง expectation ที่ตรงกัน

Summary

IT = Enabler, Maintenance = Developer

สร้าง Self-Service Platform ที่ Maintenance Deploy เองได้
Zero Config • On-Premise • AI-Powered • Data Privacy
เป้าหมาย: Maintenance deploy เองได้ 100% ใน 15 นาที